Working with Files

in Python 3

File I/O, Directory Management, Archives, Serialisation & Real-World Patterns

Part of the Series:

End-to-End GenAI & Data Science: Production-Ready Engineering

- Finding & Listing Files — os, glob, fnmatch
- File & Directory Attributes — Metadata, Size, Timestamps
- Directory Traversal — os.walk and Recursive Search
- File Operations — Copy, Move, Rename, Delete
- Archiving — Creating, Reading & Extracting ZIP Files
- File I/O — Text, CSV, XML, JSON
- Object Persistence — pickle & Real-World Model Serialisation
- Modern pathlib API — The Production-Grade Alternative

SECTION 1 — THEORY

Chapter 1: Introduction to File Operations in Python

1.1 Why File Handling Matters

Every non-trivial software system must interact with persistent storage. Whether loading a trained machine learning model, reading configuration parameters, ingesting a data pipeline's input CSV, logging application events, or archiving outputs for distribution — file operations sit at the intersection of nearly every production engineering task. In data science and GenAI contexts specifically, file handling is not a peripheral concern: it is a daily, foundational discipline.

Python's approach to file handling reflects its broader design philosophy: a rich standard library provides batteries-included support for the most common file formats and operations, while the language's context manager protocol (the with statement) makes safe, leak-free file access idiomatic and effortless.

File Type	Typical Use in Data Science / GenAI	Python Module
Text (.txt, .log)	Logs, configuration, plain prompts, reports	open() built-in
CSV (.csv)	Tabular datasets, feature matrices, results	csv, pandas
JSON (.json)	API payloads, LLM prompts, config, metadata	json
XML (.xml)	Legacy configs, document exchange, web scraping	xml.etree.ElementTree
ZIP / TAR archives	Dataset distribution, model checkpoints, backups	zipfile, tarfile, shutil
Pickle (.pkl)	Python object serialisation: models, arrays, dicts	pickle
Parquet / HDF5	Columnar dataset storage (high performance)	pandas, h5py

1.2 The Python File Handling Ecosystem

Python organises file-handling functionality across several standard-library modules, each with a distinct responsibility:

- `os` — The foundational operating-system interface: listing directories, querying file attributes, renaming, removing, and walking directory trees. OS-agnostic by design.
- `os.path` — Path manipulation utilities: joining path components, extracting extensions and basenames, checking existence and type, computing sizes.
- `pathlib` — The modern, object-oriented path API (Python 3.4+). Replaces most os.path usage with a clean, chainable interface. Preferred for all new code.
- `shutil` — High-level file operations: copy, move, and archive. Handles edge cases (cross-device moves, permission preservation) that naive os calls miss.
- `glob` — Shell-style pattern matching for filesystem paths. Supports wildcards and recursive descent.
- `fnmatch` — Low-level filename pattern matching. Used inside glob's implementation and useful for filtering pre-loaded file lists.
- `zipfile` and `tarfile` — Archive creation, inspection, and extraction for ZIP and TAR formats respectively.
- `csv` — Comma-separated value reading and writing with configurable dialects.
- `json` — Encoding and decoding JSON data structures to and from Python objects.
- `pickle` — Binary serialisation of arbitrary Python objects — the mechanism behind saved ML models and cached data structures.

> **Design Principle:** Prefer pathlib over os.path for all path manipulation in new code. It offers a cleaner API, better cross-platform behaviour, and makes code significantly more readable. The os module remains necessary for process-level operations (environment variables, process spawning) and os.walk().

1.3 The Context Manager Protocol — Safe File Handling

Before examining individual file operations, it is essential to understand the with statement — the idiomatic mechanism for safe file access in Python. Every file opened in Python must eventually be closed to release the operating system file descriptor. Failure to close files causes resource leaks and, in long-running processes, can exhaust the OS file descriptor limit.

The with statement invokes the context manager protocol: upon entering the block, the file is opened; upon exiting — regardless of whether the exit is normal or due to an exception — the file's __exit__ method is called, which closes the file handle. This guarantees cleanup even when exceptions occur.

```
# Without context manager — fragile: file never closed if exception
occurs
f = open('data.csv', 'r')
content = f.read()  # if this raises an exception, f.close() is never
called
f.close()           # resource leak risk
```

```
# With context manager — safe: file always closed, even on exception
with open('data.csv', 'r') as f:
    content = f.read()
# f is now closed automatically — guaranteed

# Multiple files in one context manager (Python 3.10+)
with open('source.txt', 'r') as src, open('dest.txt', 'w') as dst:
    dst.write(src.read())
```

File Open Mode	Symbol	Behaviour
Read (default)	r	Opens for reading. Raises FileNotFoundError if file absent.
Write	w	Opens for writing. Creates file if absent; truncates if present.
Append	a	Opens for writing at end-of-file. Creates if absent; never truncates.
Read + Write	r+	Opens for both reading and writing. File must exist.
Binary Read	rb	Read in binary mode — required for images, pickles, ZIPs.
Binary Write	wb	Write in binary mode — required for images, pickles, ZIPs.
Exclusive Create	x	Creates new file. Raises FileExistsError if file already exists.

Chapter 2: Finding & Listing Files

2.1 Listing Directory Contents with os.listdir()

The most fundamental file discovery operation is listing the contents of a directory. The os.listdir() function returns a list of the names of all entries in a directory — files, subdirectories, symlinks, and hidden files alike — as plain strings, without any path prefix.

```
import os

# List current working directory
entries = os.listdir()          # equivalent to os.listdir('.')
for entry in sorted(entries):   # sorted for deterministic output
    print(entry)
```

```
# List a specific directory
documents = os.listdir('Documents')
print(documents)
# ['budget.xlsx', 'notes.txt', 'report_2024.csv', 'presentations']
```

Important characteristics of os.listdir() to be aware of in production code:

- **Returns names only, not full paths.** To construct absolute paths, join with the directory: os.path.join(directory, name).
- **Does not recurse into subdirectories.** It lists only immediate children. For recursive traversal, use os.walk() (Chapter 4).
- **Includes hidden files.** On Unix-like systems, files beginning with a dot (.) are hidden. os.listdir() includes them; filter with name.startswith('.') if needed.
- **Order is arbitrary.** The OS does not guarantee alphabetical order. Always sort() the result if ordering matters.

2.2 String Methods for File Filtering

Once a directory listing is obtained, the names are plain Python strings. Python's string methods provide an efficient toolkit for filtering and inspecting filenames before performing any I/O operations:

```
import os

# Get all entries and filter using string methods
entries = os.listdir('datasets')

# Filter by file extension
csv_files  = [f for f in entries if f.endswith('.csv')]
json_files = [f for f in entries if f.endswith('.json')]
data_files = [f for f in entries if f.endswith(('.csv', '.parquet',
'.json'))]

# Filter by name prefix
report_files = [f for f in entries if f.startswith('report_')]

# Filter by substring anywhere in name
year_files = [f for f in entries if '2024' in f]

# Combine conditions
report_csvs = [f for f in entries
               if f.startswith('report_') and f.endswith('.csv')]

# Extract base name and extension
filename = 'report_2024_Q3.csv'
name, ext = os.path.splitext(filename)
print(f'Name: {name}, Extension: {ext}')  # Name: report_2024_Q3,
Extension: .csv
```

Practical Note: The endswith() method accepts a tuple of suffixes — a frequently overlooked feature that eliminates the need for any() or multiple or conditions when checking for several extensions simultaneously.

2.3 Pattern Matching with fnmatch

The fnmatch module implements Unix shell-style wildcard pattern matching against filenames. It is the low-level primitive that powers glob, and is useful when you already have a list of filenames (from os.listdir() or another source) and need to filter it without performing a filesystem scan.

Pattern	Matches	Example Match
*	Any sequence of characters (including none)	*.csv matches report.csv, data.csv, .csv
?	Any single character	data_?.csv matches data_1.csv but not data_10.csv
[seq]	Any character in the sequence	[abc]*.txt matches apple.txt, banana.txt
[!seq]	Any character NOT in the sequence	[!d]*.csv matches report.csv but not data.csv

```
import fnmatch
import os

# Single match check
print(fnmatch.fnmatch('report.csv', '*.csv'))        # True
print(fnmatch.fnmatch('report.CSV', '*.csv'))        # False (case-
sensitive on Unix)
print(fnmatch.fnmatch('data_2024_Q3.csv', 'data_*')) # True

# Filter a list of filenames against a pattern
all_files = os.listdir('datasets')
csv_files = fnmatch.filter(all_files, '*.csv')
print(csv_files)  # ['data_2024.csv', 'report_q3.csv', ...]

# Case-insensitive matching on Windows-style names
print(fnmatch.fnmatchcase('Report.CSV', '*.csv'))  # False (explicit
case-sensitive)

# Advanced multi-pattern filtering
patterns = ['*.csv', 'report_*', 'data_2024_*']
matched = [f for f in all_files
           if any(fnmatch.fnmatch(f, p) for p in patterns)]
```

2.4 Pattern Matching with glob

The glob module extends fnmatch's capabilities by performing the pattern matching directly against the filesystem, combining directory traversal and filtering in a single operation. It is the preferred tool when you know the pattern of files you want and their location:

```
import glob

# Find all CSV files in the current directory
csv_files = glob.glob('*.csv')
print(csv_files)  # ['data.csv', 'report.csv', 'sales_2024.csv']

# Find CSV files in a specific subdirectory
dataset_csvs = glob.glob('datasets/*.csv')

# Find all Python files in the current directory only
py_files = glob.glob('*.py')

# Recursive search: ** matches any directory depth
# IMPORTANT: must set recursive=True for ** to work
all_csvs_recursive = glob.glob('**/*.csv', recursive=True)
print(all_csvs_recursive)
# Finds: data.csv, subdir/more_data.csv, subdir/nested/archive.csv, ...

# Find all JSON and CSV files (requires two calls or itertools.chain)
import itertools
data_files = list(itertools.chain(
    glob.glob('**/*.csv', recursive=True),
    glob.glob('**/*.json', recursive=True)
))

# Using pathlib.Path.glob() — the modern alternative
from pathlib import Path
csv_files_modern = list(Path('datasets').glob('*.csv'))
csv_files_recursive = list(Path('.').rglob('*.csv'))  # rglob = recursive
glob
```

pathlib Advantage: Path.rglob('*.csv') is the modern, preferred equivalent of glob.glob('**/*.csv', recursive=True). It is more readable, returns Path objects instead of strings (enabling method chaining), and integrates naturally with the rest of the pathlib API.

Chapter 3: File & Directory Attributes

3.1 Querying File Metadata

Before processing a file, production code often needs to inspect its metadata: size (to decide whether to load it into memory or stream it), modification timestamp (to determine if a cached

result is stale), and type (to distinguish files from directories or symlinks). Python provides these through os.path, os.stat(), and pathlib.

```
import os
import datetime

filepath = 'datasets/training_data.csv'

# ── os.path convenience functions ──────────────────────────
print(os.path.exists(filepath))       # True / False
print(os.path.isfile(filepath))       # True if it's a regular file
print(os.path.isdir(filepath))        # True if it's a directory
print(os.path.getsize(filepath))      # Size in bytes
print(os.path.abspath(filepath))      # Absolute path
print(os.path.basename(filepath))     # 'training_data.csv'
print(os.path.dirname(filepath))      # 'datasets'
print(os.path.splitext(filepath))     # ('datasets/training_data',
'.csv')

# ── os.stat() — low-level stat structure ───────────────────
stat = os.stat(filepath)
print(f'Size          : {stat.st_size:,} bytes')
print(f'Last modified :
{datetime.datetime.fromtimestamp(stat.st_mtime)}')
print(f'Last accessed :
{datetime.datetime.fromtimestamp(stat.st_atime)}')
print(f'Permissions   : {oct(stat.st_mode)}')

# ── pathlib equivalent — cleaner and preferred ─────────────
from pathlib import Path
p = Path(filepath)
stat2 = p.stat()
print(f'Stem    : {p.stem}')    # 'training_data'  (name without
extension)
print(f'Suffix  : {p.suffix}')  # '.csv'
print(f'Parent  : {p.parent}')  # 'datasets'
print(f'Size    : {stat2.st_size:,} bytes')
print(f'Modified:
{datetime.datetime.fromtimestamp(stat2.st_mtime).isoformat()}')
```

os.path Function	pathlib Equivalent	Returns
os.path.exists(p)	Path(p).exists()	bool — file or directory exists
os.path.isfile(p)	Path(p).is_file()	bool — is a regular file
os.path.isdir(p)	Path(p).is_dir()	bool — is a directory
os.path.getsize(p)	Path(p).stat().st_size	int — size in bytes
os.path.abspath(p)	Path(p).resolve()	Absolute path string / Path object
os.path.basename(p)	Path(p).name	Filename with extension

os.path.splitext(p)[0]	Path(p).stem	Filename without extension
os.path.splitext(p)[1]	Path(p).suffix	Extension including dot (.csv)
os.path.dirname(p)	Path(p).parent	Parent directory
os.path.join(a, b)	Path(a) / b	Combined path

3.2 Building a File Inventory (Applied Example)

A common production pattern in data engineering is auditing a directory of data files to build an inventory: enumerating all files, recording their sizes, and identifying which ones have been recently modified. This is the foundation of incremental processing pipelines.

```
from pathlib import Path
import datetime

def build_file_inventory(directory: str, pattern: str = '*') ->
list[dict]:
    """
    Build a metadata inventory of all files matching a pattern.

    Parameters
    ----------
    directory : str
        Root directory to scan.
    pattern : str
        Glob pattern for file selection (default: all files).

    Returns
    -------
    list[dict]
        List of dicts with keys: name, path, size_bytes, size_mb,
        modified, extension.
    """
    root = Path(directory)
    inventory = []

    for path in sorted(root.rglob(pattern)):
        if not path.is_file():
            continue
        stat = path.stat()
        inventory.append({
            'name':       path.name,
            'path':       str(path),
            'size_bytes': stat.st_size,
            'size_mb':    round(stat.st_size / 1_048_576, 3),
            'modified':
datetime.datetime.fromtimestamp(stat.st_mtime).isoformat(),
            'extension':  path.suffix.lower(),
        })

    return inventory
```

```
# Usage:
# inventory = build_file_inventory('data/', '*.csv')
# for item in inventory:
#     print(f"{item['name']:<40} {item['size_mb']:>8.3f} MB
{item['modified']}")
```

Chapter 4: Directory Traversal with os.walk()

4.1 Understanding os.walk()

While os.listdir() provides a flat view of a single directory, os.walk() performs a recursive tree walk, visiting every subdirectory in turn. At each directory it visits, it yields a 3-tuple: the current directory path, the list of immediate subdirectory names it contains, and the list of regular file names it contains.

```
import os

# os.walk() yields: (dirpath, subdirectory_names, file_names)
for root, dirs, files in os.walk('.'):
    print(f'Directory : {root}')
    print(f'  Subdirs : {dirs}')
    print(f'  Files   : {files}')
    print()

# Example output for a project directory:
# Directory : .
#   Subdirs : ['data', 'models', 'notebooks']
#   Files   : ['README.md', 'requirements.txt', 'train.py']
#
# Directory : ./data
#   Subdirs : ['raw', 'processed']
#   Files   : ['schema.json']
#
# Directory : ./data/raw
#   Subdirs : []
#   Files   : ['train.csv', 'val.csv', 'test.csv']
```

4.2 Practical os.walk() Patterns

The raw 3-tuple from os.walk() becomes powerful when combined with path joining and filtering. Below are the canonical patterns used in production data engineering code:

```
import os

# ── Pattern 1: Collect all files of a given type recursively ──
def find_files(root_dir: str, extension: str) -> list[str]:
    """Return absolute paths of all files with the given extension."""
    results = []
```

```
    for dirpath, _, filenames in os.walk(root_dir):
        for fname in filenames:
            if fname.endswith(extension):
                results.append(os.path.join(dirpath, fname))
    return results

csv_paths = find_files('data/', '.csv')


# ── Pattern 2: Compute total size of a directory tree ──────────
def directory_size(root_dir: str) -> int:
    """Return total byte count of all files under root_dir."""
    total = 0
    for dirpath, _, filenames in os.walk(root_dir):
        for fname in filenames:
            fpath = os.path.join(dirpath, fname)
            total += os.path.getsize(fpath)
    return total

total_bytes = directory_size('data/')
print(f'Total: {total_bytes / 1_048_576:.2f} MB')


# ── Pattern 3: Prune specific subdirectories from traversal ────
# Modifying `dirs` IN-PLACE controls which subdirs os.walk visits next
for root, dirs, files in os.walk('.'):
    dirs[:] = [d for d in dirs if d not in {'.git', '__pycache__',
'node_modules'}]
    for fname in files:
        print(os.path.join(root, fname))
```

Advanced Tip: Modifying the dirs list in-place (dirs[:] = [...]) inside the os.walk() loop prunes which subdirectories the walk will descend into. This is essential for excluding version control directories, cache directories, and virtual environments from scans, making the traversal both faster and more correct.

Chapter 5: Core File Operations — Copy, Move, Rename, Delete

5.1 Copying Files with shutil

The shutil module (shell utilities) provides high-level file operations that handle the cross-platform complexity that raw os calls leave to the developer. shutil.copy() is the primary function for duplicating files:

```
import shutil
import os
```

```
# ── shutil.copy() — copies file content and permissions ────────
shutil.copy('report.txt', 'backup/report.txt')   # copy to specific path
shutil.copy('report.txt', 'backup/')              # copy into directory,
same name

# ── shutil.copy2() — also preserves timestamps ─────────────────
shutil.copy2('report.txt', 'archive/report.txt')  # preserves mtime,
atime

# ── shutil.copytree() — recursively copy an entire directory ───
shutil.copytree('models/', 'models_backup/')

# ── Copying with error handling (production pattern) ───────────
from pathlib import Path

def safe_copy(src: str, dst: str) -> bool:
    """Copy src to dst; return True on success, False on failure."""
    try:
        Path(dst).parent.mkdir(parents=True, exist_ok=True)  # ensure dst
dir
        shutil.copy2(src, dst)
        return True
    except (FileNotFoundError, PermissionError, OSError) as e:
        print(f'Copy failed: {e}')
        return False
```

Function	Content	Metadata (timestamps)	Permissions	Notes
shutil.copy(src, dst)	Yes	No	Yes	Most common; fast
shutil.copy2(src, dst)	Yes	Yes	Yes	Preferred for backups
shutil.copyfile(src, dst)	Yes	No	No	Content only; dst must be a path not directory
shutil.copytree(src, dst)	Yes (recursive)	Yes	Yes	Copies entire directory tree

5.2 Moving Files with shutil.move()

shutil.move() transfers a file or directory from one location to another. If the source and destination are on the same filesystem, the operation is an atomic rename (instantaneous). If they are on different filesystems (e.g., moving from one drive to another), shutil.move() falls back to a copy-then-delete sequence.

```
import shutil
```

```
# Move a file to a different directory (same filename retained)
shutil.move('report.txt', 'archive/')

# Move and rename in one operation
shutil.move('report.txt', 'archive/report_2024_Q3.txt')

# Move an entire directory
shutil.move('temp_models/', 'production_models/')

# ── Production pattern: sort incoming files by type ──────────────
import os
from pathlib import Path

def sort_by_extension(source_dir: str, target_dir: str) -> None:
    """Move files in source_dir into type-named subdirs of target_dir."""
    for fpath in Path(source_dir).iterdir():
        if not fpath.is_file():
            continue
        ext = fpath.suffix.lstrip('.').lower() or 'no_extension'
        dest = Path(target_dir) / ext
        dest.mkdir(parents=True, exist_ok=True)
        shutil.move(str(fpath), str(dest / fpath.name))
        print(f'Moved {fpath.name} → {dest}/')
```

5.3 Renaming Files with os.rename()

os.rename() renames (or moves within the same filesystem) a file or directory. On most operating systems, this is an atomic operation when the source and destination are on the same filesystem — the rename either completes fully or not at all, making it safe for concurrent systems.

```
import os
from pathlib import Path

# Simple rename
os.rename('old_name.txt', 'new_name.txt')

# pathlib equivalent — equally idiomatic
Path('old_name.txt').rename('new_name.txt')

# ── Batch rename: add date prefix to all CSVs in a directory ─────
import datetime

def add_date_prefix(directory: str, extension: str = '.csv') -> None:
    """Rename all files of given type to include today's date as
prefix."""
    today = datetime.date.today().strftime('%Y%m%d')
    for path in Path(directory).iterdir():
        if path.is_file() and path.suffix == extension:
            new_name = path.parent / f'{today}_{path.name}'
            path.rename(new_name)
            print(f'Renamed: {path.name}  →  {new_name.name}')

# ── Safe rename: only rename if destination does not already exist
```

```
def safe_rename(src: str, dst: str) -> bool:
    src_path, dst_path = Path(src), Path(dst)
    if dst_path.exists():
        print(f'Rename aborted: {dst} already exists.')
        return False
    src_path.rename(dst_path)
    return True
```

5.4 Deleting Files and Directories

File deletion is irreversible in most operating system configurations (the file does not pass through a recycle bin when deleted via Python). Production code must therefore apply defensive checks and, where possible, prefer soft deletion (moving to a trash/archive directory) over permanent removal.

```
import os
import shutil
from pathlib import Path

# ── Delete a single file ──────────────────────────────────
os.remove('temp.txt')           # raises FileNotFoundError if absent
Path('temp.txt').unlink()       # pathlib equivalent
Path('temp.txt').unlink(missing_ok=True)  # silent if absent (Python
3.8+)

# ── Delete an empty directory ─────────────────────────────
os.rmdir('empty_dir/')          # raises OSError if directory is not
empty
Path('empty_dir/').rmdir()      # pathlib equivalent

# ── Delete a directory tree (recursive, irreversible!) ────
shutil.rmtree('old_models/')    # permanently removes everything

# ── Production pattern: safe deletion with archiving ──────
def soft_delete(filepath: str, trash_dir: str = '.trash') -> None:
    """Move file to a trash directory instead of permanently deleting."""
    src = Path(filepath)
    trash = Path(trash_dir)
    trash.mkdir(parents=True, exist_ok=True)
    dest = trash / src.name
    if dest.exists():
        # Avoid collision by appending a counter
        counter = 1
        while dest.exists():
            dest = trash / f'{src.stem}_{counter}{src.suffix}'
            counter += 1
    shutil.move(str(src), str(dest))
    print(f'Soft-deleted: {filepath}  →  {dest}')
```

Safety Rule: Never call shutil.rmtree() in production without first confirming the path is inside the expected directory tree. A bug that causes the path argument to resolve to '/' or

'C:\\' would wipe the entire filesystem. Always validate with Path(target).is_relative_to(expected_root) before deletion.

Chapter 6: Archiving Files — ZIP Operations

6.1 The zipfile Module

The zipfile module provides a complete interface for creating, reading, and extracting ZIP archives. ZIP is one of the most universally supported archive formats — understood by every major operating system and supported by cloud storage, email clients, and virtually all software distribution platforms.

In data science contexts, ZIP archives serve several important roles:

- Dataset distribution: large CSV or JSON datasets are commonly distributed as ZIP files to reduce download size.
- Model checkpoint packaging: saving a model together with its configuration, vocabulary, and tokeniser files in a single archive.
- Backup automation: scripted archiving of processed outputs and experiment results.
- Data ingestion: many public datasets (Kaggle, UCI ML Repository) are delivered as ZIP files that must be extracted programmatically.

6.2 Creating ZIP Archives

```
from zipfile import ZipFile, ZIP_DEFLATED
from pathlib import Path

# ── Basic: add a single file to a ZIP ──────────────────────────
with ZipFile('archive.zip', 'w') as zipf:
    zipf.write('report.txt')          # stored with its original name
    zipf.write('data.csv', 'data/data.csv')  # store under a different
path

# ── With compression: ZIP_DEFLATED uses zlib for size reduction ─
with ZipFile('compressed.zip', 'w', compression=ZIP_DEFLATED,
compresslevel=9) as zipf:
    zipf.write('report.txt')

# ── Archive an entire directory tree ───────────────────────────
def zip_directory(source_dir: str, output_zip: str) -> None:
    """Create a ZIP archive of an entire directory tree."""
    source = Path(source_dir)
    with ZipFile(output_zip, 'w', compression=ZIP_DEFLATED) as zipf:
        for path in source.rglob('*'):
            if path.is_file():
                # Store relative path inside the ZIP to preserve
structure
                arcname = path.relative_to(source.parent)
```

```
                zipf.write(path, arcname)
                print(f'  Added: {arcname}')

zip_directory('experiment_results/', 'experiment_2024_Q3.zip')

# ── shutil convenience: archive entire tree in one call ──────────
import shutil
shutil.make_archive('my_archive', 'zip', 'source_directory/')
```

6.3 Reading & Inspecting ZIP Archives

```
from zipfile import ZipFile

# ── List all files inside a ZIP without extracting ───────────────
with ZipFile('archive.zip') as zipf:
    print(zipf.namelist())  # ['report.txt', 'data/data.csv',
'config.json']

    # Inspect metadata of each member
    for info in zipf.infolist():
        print(f'{info.filename:<40} {info.file_size:>10,} bytes'
              f'  compressed: {info.compress_size:>8,} bytes')

# ── Read a specific file from inside the ZIP without extracting ──
with ZipFile('dataset.zip') as zipf:
    with zipf.open('data/train.csv') as csv_file:
        import io
        import csv
        reader = csv.DictReader(io.TextIOWrapper(csv_file, encoding='utf-
8'))
        first_row = next(reader)
        print(first_row)

# ── Check if a ZIP is valid ──────────────────────────────────────
with ZipFile('archive.zip') as zipf:
    corrupt = zipf.testzip()   # returns name of first bad file, or None
    if corrupt:
        print(f'Corrupt file found: {corrupt}')
    else:
        print('Archive is valid.')
```

6.4 Extracting ZIP Archives

```
from zipfile import ZipFile
from pathlib import Path

# ── Extract all files to a directory ─────────────────────────────
with ZipFile('archive.zip') as zipf:
    zipf.extractall('output/')   # creates output/ if it does not exist

# ── Extract a single named file ──────────────────────────────────
with ZipFile('archive.zip') as zipf:
    zipf.extract('data/train.csv', 'output/')  # → output/data/train.csv
```

```
# ── Extract selectively: only CSV files ──────────────────────
with ZipFile('dataset.zip') as zipf:
    csv_members = [name for name in zipf.namelist() if
name.endswith('.csv')]
    for member in csv_members:
        zipf.extract(member, 'output/csv/')
        print(f'Extracted: {member}')

# ── Security: zip-slip path traversal defence ────────────────
# IMPORTANT: malicious ZIPs can contain paths like '../../etc/passwd'
# Always validate extracted paths stay within the output directory
def safe_extract(zip_path: str, output_dir: str) -> None:
    """Extract ZIP while defending against path traversal attacks."""
    output = Path(output_dir).resolve()
    with ZipFile(zip_path) as zipf:
        for member in zipf.infolist():
            target = (output / member.filename).resolve()
            if not str(target).startswith(str(output)):
                raise ValueError(f'Path traversal detected:
{member.filename}')
            zipf.extract(member, output_dir)
```

Security Warning: Always validate extracted paths when processing ZIP files from untrusted sources. The 'zip-slip' vulnerability allows malicious archives to overwrite arbitrary files on the system by embedding path traversal sequences (../../) in filenames. The safe_extract() function above demonstrates the correct defence.

Chapter 7: Reading and Writing Files

7.1 Text Files

Text files are the most fundamental file format — human-readable, portable across operating systems, and supported natively by Python's built-in open() function. They are the default choice for logs, configuration files, plain-text reports, and LLM prompt templates.

```
# ── Reading text files ──────────────────────────────────────

# Read entire file into a string
with open('notes.txt', 'r', encoding='utf-8') as f:
    content = f.read()

# Read line by line (memory-efficient for large files)
with open('large_log.txt', 'r', encoding='utf-8') as f:
    for line_number, line in enumerate(f, start=1):
        line = line.rstrip('\n')   # strip trailing newline
        print(f'{line_number:5d}: {line}')

# Read all lines into a list
with open('notes.txt', 'r', encoding='utf-8') as f:
```

```
    lines = f.readlines()   # retains '\n' at end of each line
    lines = [l.strip() for l in lines]  # clean version

# ── Writing text files ──────────────────────────────────────

# Write (creates or overwrites)
with open('output.txt', 'w', encoding='utf-8') as f:
    f.write('First line\n')
    f.write('Second line\n')
    f.writelines(['Third\n', 'Fourth\n'])  # write multiple lines at once

# Append to existing file (creates if absent)
with open('application.log', 'a', encoding='utf-8') as f:
    import datetime
    f.write(f'{datetime.datetime.now().isoformat()} INFO Training
complete\n')
```

Best Practice: Always specify encoding='utf-8' explicitly when opening text files. Python's default encoding is platform-dependent (cp1252 on Windows, UTF-8 on Linux/macOS), causing files written on one platform to be unreadable on another. Explicit UTF-8 ensures portability across all environments.

7.2 CSV Files

CSV (Comma-Separated Values) is the lingua franca of tabular data exchange. Despite its simplicity, correctly handling CSV files requires attention to quoting rules, delimiter variations, header rows, encoding, and missing values — all of which Python's csv module handles correctly.

```
import csv

# ── Reading CSV with DictReader (each row is a dict) ────────
with open('data.csv', 'r', encoding='utf-8', newline='') as f:
    reader = csv.DictReader(f)
    print(f'Columns: {reader.fieldnames}')
    for row in reader:
        print(row)   # {'name': 'Alice', 'age': '30', 'score': '95.2'}

# ── Reading CSV with reader (each row is a list) ────────────
with open('data.csv', 'r', encoding='utf-8', newline='') as f:
    reader = csv.reader(f)
    headers = next(reader)  # consume header row
    for row in reader:
        print(row)  # ['Alice', '30', '95.2']

# ── Writing CSV with DictWriter ─────────────────────────────
results = [
    {'model': 'RandomForest', 'accuracy': 0.923, 'f1': 0.918},
    {'model': 'XGBoost',      'accuracy': 0.941, 'f1': 0.937},
    {'model': 'NeuralNet',    'accuracy': 0.956, 'f1': 0.952},
]
```

```
with open('model_results.csv', 'w', encoding='utf-8', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=['model', 'accuracy', 'f1'])
    writer.writeheader()
    writer.writerows(results)

# ── Handling tab-separated and custom delimiters ──────────────────
with open('data.tsv', 'r', encoding='utf-8', newline='') as f:
    reader = csv.reader(f, delimiter='\t')
    for row in reader:
        print(row)
```

Data Science Note: For production data science, pandas.read_csv() is preferable over the csv module when working with large datasets, as it handles type inference, missing values, date parsing, and memory optimisation automatically. Use the csv module for small files, streaming large files, or when pandas is not available.

7.3 JSON Files

JSON (JavaScript Object Notation) is the dominant data interchange format in modern software — used by virtually every REST API, configuration system, and LLM prompt framework. Python's json module provides seamless conversion between JSON text and Python's native data structures.

```
import json
from pathlib import Path

# ── Reading JSON ──────────────────────────────────────────────────
with open('config.json', 'r', encoding='utf-8') as f:
    config = json.load(f)      # parse JSON → Python dict/list
    print(config['model_name'])  # access as a normal dict

# Parse JSON from a string (e.g., an API response body)
json_string = '{"name": "Alice", "score": 95.2}'
obj = json.loads(json_string)  # string → Python object

# ── Writing JSON ──────────────────────────────────────────────────
experiment = {
    'run_id': 'exp_2024_q3_001',
    'model': 'GPT-4-fine-tuned',
    'hyperparameters': {'lr': 1e-4, 'epochs': 10, 'batch_size': 32},
    'metrics': {'train_loss': 0.142, 'val_loss': 0.187, 'accuracy':
0.941},
    'timestamp': '2024-11-15T14:32:00Z',
}

with open('experiment_log.json', 'w', encoding='utf-8') as f:
    json.dump(experiment, f, indent=4, ensure_ascii=False)

# ── Serialise to JSON string (for API payloads, logging) ──────────
```

```
json_str = json.dumps(experiment, indent=2)
print(json_str)

# ── Handling non-serialisable types with a custom encoder ────────
import datetime
import numpy as np

class CustomEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime.datetime):
            return obj.isoformat()
        if isinstance(obj, np.ndarray):
            return obj.tolist()
        if isinstance(obj, np.integer):
            return int(obj)
        if isinstance(obj, np.floating):
            return float(obj)
        return super().default(obj)

with open('results.json', 'w') as f:
    json.dump(results_with_numpy, f, cls=CustomEncoder, indent=4)
```

7.4 XML Files

XML (eXtensible Markup Language) stores data in a hierarchical, tag-delimited structure. While largely superseded by JSON in modern API design, XML remains prevalent in legacy enterprise systems, document formats (DOCX, SVG, Android resources), RSS feeds, and certain scientific data formats (SBML, CML).

```
import xml.etree.ElementTree as ET

# ── Reading XML ──────────────────────────────────────────────────
# Sample XML:
# <students>
#   <student id='1'>
#     <name>Sarah</name>
#     <grade>A</grade>
#   </student>
# </students>

tree = ET.parse('students.xml')
root = tree.getroot()
print(f'Root tag: {root.tag}')  # 'students'

for student in root.findall('student'):
    student_id = student.get('id')      # attribute access
    name  = student.find('name').text   # child element text
    grade = student.find('grade').text
    print(f'ID: {student_id}, Name: {name}, Grade: {grade}')

# ── Writing XML ──────────────────────────────────────────────────
root = ET.Element('experiment_results')
```

```
for model_name, accuracy in [('RandomForest', 0.923), ('XGBoost',
0.941)]:
    model_el = ET.SubElement(root, 'model')
    model_el.set('name', model_name)
    acc_el = ET.SubElement(model_el, 'accuracy')
    acc_el.text = str(accuracy)

tree = ET.ElementTree(root)
ET.indent(tree, space='  ')   # pretty-print (Python 3.9+)
tree.write('results.xml', encoding='utf-8', xml_declaration=True)
```

Chapter 8: Object Persistence with pickle

8.1 What is pickle?

The pickle module implements Python's native binary serialisation protocol — the mechanism by which arbitrary Python objects (including trained machine learning models, NumPy arrays, scikit-learn pipelines, PyTorch state dictionaries, and custom class instances) are converted to a binary byte stream that can be written to disk and later restored to an identical in-memory representation.

This capability is foundational to machine learning workflows: a model trained over hours or days should not need to be retrained every time it is needed. Persistence allows the trained model to be serialised once and loaded instantly on demand.

Format	Human-readable	Python Objects	Cross-language	ML Model Support	Best For
pickle	No (binary)	Any Python object	No	Full support	Python-internal persistence
JSON	Yes	dict, list, str, int, float, bool, None	Yes	Limited (no arrays)	API payloads, config files
joblib	No (binary)	NumPy arrays + estimators	No	Optimised for sklearn	scikit-learn models
ONNX	No (binary)	Model architecture + weights	Yes (cross-framework)	Full support	Cross-platform model deployment

8.2 Basic pickle Usage

```
import pickle

# ── Serialise (pickle) a Python object to disk ──────────────────
# Any Python object: dict, list, class instance, trained model...
```

```
vocabulary = {'hello': 1, 'world': 2, 'python': 3, 'data': 4}

with open('vocabulary.pkl', 'wb') as f:   # 'wb' = write binary
    pickle.dump(vocabulary, f, protocol=pickle.HIGHEST_PROTOCOL)

# ── Deserialise (unpickle) from disk ──────────────────────────
with open('vocabulary.pkl', 'rb') as f:   # 'rb' = read binary
    loaded_vocab = pickle.load(f)

print(loaded_vocab)  # {'hello': 1, 'world': 2, 'python': 3, 'data': 4}
print(type(loaded_vocab))  # <class 'dict'>

# ── Pickle directly to/from bytes (for in-memory or network use) ─
serialised_bytes = pickle.dumps(vocabulary)
restored = pickle.loads(serialised_bytes)
```

8.3 Persisting Trained Machine Learning Models

```
import pickle
from pathlib import Path
import datetime

# ── Example: train and persist a scikit-learn model ─────────────
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Train the model
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
accuracy = accuracy_score(y_test, model.predict(X_test))
print(f'Training accuracy: {accuracy:.4f}')

# 2. Persist the trained model with metadata
model_artifact = {
    'model':       model,
    'feature_names': ['sepal_length', 'sepal_width', 'petal_length',
'petal_width'],
    'class_names': ['setosa', 'versicolor', 'virginica'],
    'accuracy':    accuracy,
    'trained_at':  datetime.datetime.now().isoformat(),
}

model_path = Path('models/iris_rf_v1.pkl')
model_path.parent.mkdir(parents=True, exist_ok=True)

with open(model_path, 'wb') as f:
    pickle.dump(model_artifact, f, protocol=pickle.HIGHEST_PROTOCOL)
print(f'Model saved to {model_path}  ({model_path.stat().st_size:,}
bytes)')
```

```
# 3. Load the model for inference
with open(model_path, 'rb') as f:
    artifact = pickle.load(f)

loaded_model = artifact['model']
new_data = [[5.1, 3.5, 1.4, 0.2]]   # a new Iris measurement
prediction = loaded_model.predict(new_data)
print(f'Prediction: {artifact["class_names"][prediction[0]]}')
```

> **Security Warning:** Never unpickle data from untrusted sources. The pickle format allows arbitrary Python code to execute during deserialisation. A malicious pickle file can gain full control of the Python process. For model sharing across teams or organisations, prefer joblib (for sklearn), torch.save/load (for PyTorch), or ONNX (for cross-framework deployment).

8.4 joblib — The Preferred Choice for Large ML Artifacts

While pickle works for all Python objects, joblib is the recommended serialisation library for scikit-learn models and NumPy-heavy objects. joblib uses memory-mapped files for large arrays, which can be dramatically faster for models containing large weight matrices or preprocessing pipelines.

```
# pip install joblib  (included with scikit-learn)
import joblib

# Save — identical API to pickle, but faster for large NumPy arrays
joblib.dump(model, 'models/iris_rf_v1.joblib')

# Load
loaded_model = joblib.load('models/iris_rf_v1.joblib')

# joblib also supports compression
joblib.dump(model, 'models/iris_rf_v1_compressed.joblib.gz', compress=3)
```

8.5 The pathlib Module — The Modern, Preferred API

Having introduced os and os.path throughout this chapter for foundational understanding, we now present pathlib.Path as the preferred modern replacement for the majority of path-manipulation and file-operation tasks. pathlib was introduced in Python 3.4 (PEP 428) and has been the recommended approach since Python 3.6.

```
from pathlib import Path

# ── Path construction ──────────────────────────────────
p = Path('data') / 'processed' / 'train.csv'  # cross-platform path
joining
print(p)  # data/processed/train.csv  (or data\\processed\\train.csv on
Windows)
```

```
# ── Path inspection ──────────────────────────────────────────
print(p.name)      # 'train.csv'
print(p.stem)      # 'train'
print(p.suffix)    # '.csv'
print(p.parent)    # data/processed
print(p.parts)     # ('data', 'processed', 'train.csv')
print(p.is_absolute())
print(p.resolve())  # absolute path

# ── File reading and writing ─────────────────────────────────
# Read entire file as string
text = Path('notes.txt').read_text(encoding='utf-8')

# Write entire file from string (creates or overwrites)
Path('output.txt').write_text('Hello, Python!', encoding='utf-8')

# Read/write binary
data = Path('model.pkl').read_bytes()
Path('model_copy.pkl').write_bytes(data)

# ── Directory operations ─────────────────────────────────────
Path('output/models/v2').mkdir(parents=True, exist_ok=True)  # create
nested dirs

# Iterate directory contents
for child in Path('data').iterdir():
    print(child.name, '(dir)' if child.is_dir() else
f'{child.stat().st_size:,}B')

# Recursive glob
all_csvs = list(Path('data').rglob('*.csv'))

# ── Deletion ─────────────────────────────────────────────────
Path('temp.txt').unlink(missing_ok=True)    # delete file, silent if
absent
Path('empty_dir').rmdir()                   # delete empty directory
```

Chapter 9: Quick Reference — File Operations

Operation	os / shutil API	pathlib API
List directory	os.listdir(path)	Path(path).iterdir()
Recursive file search	os.walk(path)	Path(path).rglob(pattern)
Pattern file search	glob.glob(pattern)	Path(path).glob(pattern)
File exists	os.path.exists(p)	Path(p).exists()
File size	os.path.getsize(p)	Path(p).stat().st_size
Create directories	os.makedirs(p, exist_ok=True)	Path(p).mkdir(parents=True, exist_ok=True)

Copy file	shutil.copy2(src, dst)	— (use shutil)
Move file	shutil.move(src, dst)	Path(src).rename(dst) or shutil.move
Rename file	os.rename(old, new)	Path(old).rename(new)
Delete file	os.remove(p)	Path(p).unlink(missing_ok=True)
Read text file	open(p).read()	Path(p).read_text(encoding='utf-8')
Write text file	open(p,'w').write(t)	Path(p).write_text(t, encoding='utf-8')
Join path components	os.path.join(a, b)	Path(a) / b
Get file extension	os.path.splitext(p)[1]	Path(p).suffix

SECTION 2 — PRACTICAL & CODING

Chapter 10: Hands-On Notebook — File Operations in Practice

The following cells constitute a complete, executable Jupyter notebook. Each cell builds on the previous, demonstrating real-world file operation patterns directly applicable to data science and GenAI project workflows. Execute cells sequentially in a Jupyter environment.

Cell 1 — Environment Setup & Directory Scaffolding

```
# Cell 1: Set up a realistic project directory structure for
demonstration
from pathlib import Path
import json, csv, pickle, shutil, os

# ── Create a realistic project skeleton ──────────────────────
project_root = Path('file_demo_project')

# Define directory structure
dirs_to_create = [
    project_root / 'data' / 'raw',
    project_root / 'data' / 'processed',
    project_root / 'models',
    project_root / 'reports',
    project_root / 'logs',
    project_root / 'archive',
]

for d in dirs_to_create:
    d.mkdir(parents=True, exist_ok=True)
    print(f'Created: {d}')

# ── Seed with sample data files ──────────────────────────────
# Sample CSV dataset
csv_data = [
    ['student_id', 'name',    'score', 'grade'],
    ['S001',       'Alice',   '92.5',  'A'],
    ['S002',       'Bob',     '78.3',  'B+'],
    ['S003',       'Carol',   '85.0',  'A-'],
    ['S004',       'David',   '61.4',  'D+'],
    ['S005',       'Eve',     '95.8',  'A+'],
]

with open(project_root / 'data' / 'raw' / 'students.csv', 'w',
newline='') as f:
```

```
    csv.writer(f).writerows(csv_data)

# Sample JSON config
config = {
    'project': 'Student Performance Analysis',
    'version': '1.0.0',
    'model': {'algorithm': 'RandomForest', 'n_estimators': 100},
    'thresholds': {'passing_score': 70.0, 'honours_score': 90.0}
}

(project_root / 'data' / 'raw' / 'config.json').write_text(
    json.dumps(config, indent=4), encoding='utf-8'
)

# Sample log file
import datetime
(project_root / 'logs' / 'pipeline.log').write_text(
    f'{datetime.datetime.now().isoformat()} INFO  Project initialised\n',
    encoding='utf-8'
)

print('\nProject structure created successfully.')
```

Explanation

This cell mirrors the first step in any production data science project: establishing a reproducible, well-structured directory hierarchy programmatically. Several key patterns are demonstrated:

- **Path('file_demo_project'):** Creates a Path object representing the project root. Using Path objects throughout (rather than string concatenation) ensures cross-platform compatibility — the / operator constructs paths correctly on both Unix (forward slash) and Windows (backslash).
- **mkdir(parents=True, exist_ok=True):** The parents=True argument creates any missing intermediate directories (equivalent to Unix mkdir -p). The exist_ok=True argument suppresses the FileExistsError if the directory already exists — essential for idempotent setup scripts that can be safely re-run.
- **csv.writer(f).writerows(csv_data):** Writes a list-of-lists to a CSV file in a single call. The newline='' argument in open() is required when using the csv module to prevent double-newline issues on Windows.
- **path.write_text(json.dumps(...), encoding='utf-8'):** The pathlib write_text() method opens, writes, and closes the file in a single call — a convenience wrapper around open(..., 'w') + f.write(). Explicit UTF-8 encoding ensures portability.

Cell 2 — File Discovery and Pattern Matching

```
# Cell 2: Discover and catalogue files using os, fnmatch, and glob
import os
import fnmatch
import glob
from pathlib import Path
```

```
project_root = Path('file_demo_project')

# ── Method 1: os.listdir() + manual filtering ──────────────────
print('=== os.listdir() ===')
raw_entries = os.listdir(project_root / 'data' / 'raw')
print(f'All entries: {raw_entries}')

csv_files = [f for f in raw_entries if f.endswith('.csv')]
print(f'CSV files : {csv_files}')

# ── Method 2: fnmatch.filter() ─────────────────────────────────
print('\n=== fnmatch.filter() ===')
all_files = os.listdir(project_root)
matched = fnmatch.filter(raw_entries, '*.json')
print(f'JSON files via fnmatch: {matched}')

# ── Method 3: glob.glob() ──────────────────────────────────────
print('\n=== glob.glob() ===')
csv_via_glob = glob.glob(str(project_root / '**' / '*.csv'),
recursive=True)
print(f'CSVs (recursive glob): {csv_via_glob}')

# ── Method 4: pathlib (preferred) ──────────────────────────────
print('\n=== pathlib.Path.rglob() ===')
for path in sorted(project_root.rglob('*')):
    indent = '  ' * (len(path.parts) - len(project_root.parts) - 1)
    kind   = '[DIR] ' if path.is_dir() else f'[FILE]
{path.stat().st_size:>6} B'
    print(f'{indent}{path.name}  {kind}')
```

Explanation

This cell demonstrates all four file-discovery approaches side-by-side, allowing direct comparison of their APIs and use cases:

- **os.listdir() + list comprehension:** The lowest-level approach. Returns names only; requires manual path joining for any subsequent operation. Useful when you already have the file list in memory and want to apply simple suffix filtering.
- **fnmatch.filter():** A one-liner replacement for the list comprehension when using wildcard patterns. It is case-sensitive on Unix (case-insensitive on Windows by default) and does not perform filesystem access — it filters an existing list.
- **glob.glob() with recursive=True:** The ** pattern combined with recursive=True performs a depth-first traversal of the entire directory tree. Note the str() wrapper — glob.glob() requires a string argument, not a Path object. This is one reason pathlib is preferred for modern code.
- **pathlib.Path.rglob():** The clean, modern approach. Returns Path objects (enabling attribute access without string manipulation), integrates naturally with other pathlib operations, and requires no str() wrapper. The tree-printing logic computes indentation from the path depth, demonstrating how Path.parts makes structural reasoning about paths straightforward.

Cell 3 — File Operations: Copy, Move, Rename, Delete

```
# Cell 3: Demonstrate copy, move, rename, and delete operations
import shutil
from pathlib import Path

project_root = Path('file_demo_project')
raw_csv = project_root / 'data' / 'raw' / 'students.csv'

# ── Copy: backup before processing ──────────────────────────
processed_dir = project_root / 'data' / 'processed'
backup_csv    = project_root / 'archive' / 'students_backup.csv'

shutil.copy2(str(raw_csv), str(backup_csv))
print(f'Copied to backup: {backup_csv}')
print(f'Original exists : {raw_csv.exists()}')
print(f'Backup exists   : {backup_csv.exists()}')

# ── Process the CSV and write to processed/ ─────────────────
import csv
rows = []
with open(raw_csv, 'r', encoding='utf-8', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        row['score'] = float(row['score'])   # type conversion
        row['status'] = 'PASS' if row['score'] >= 70 else 'FAIL'
        rows.append(row)

processed_csv = processed_dir / 'students_processed.csv'
with open(processed_csv, 'w', encoding='utf-8', newline='') as f:
    writer = csv.DictWriter(f,
fieldnames=['student_id','name','score','grade','status'])
    writer.writeheader()
    writer.writerows(rows)

print(f'\nProcessed CSV written: {processed_csv}')

# ── Rename: add date stamp ──────────────────────────────────
import datetime
date_stamp = datetime.date.today().strftime('%Y%m%d')
stamped_name = processed_dir / f'students_processed_{date_stamp}.csv'
processed_csv.rename(stamped_name)
print(f'Renamed to: {stamped_name.name}')

# ── Move: move log to archive after processing ──────────────
log_file  = project_root / 'logs' / 'pipeline.log'
arch_log  = project_root / 'archive' / 'pipeline.log'
shutil.move(str(log_file), str(arch_log))
print(f'Log moved to archive: {arch_log}')
```

Explanation

This cell enacts a complete, realistic data processing pipeline using file operations as the connective tissue between stages:

- **shutil.copy2() before processing:** A production best practice — always back up the raw data before transforming it. copy2() preserves the file's modification timestamp alongside its contents, which is important for audit trails. The original file remains untouched; the backup is our safety net.
- **Processing loop with type conversion:** The float() conversion of the score field illustrates the most common data quality issue in CSV files: all values are read as strings. Explicit conversion within the loop is the correct pattern; it raises a ValueError immediately if the data is malformed, rather than silently propagating a wrong type into downstream calculations.
- **Path.rename() for date-stamping:** Appending a date stamp to processed outputs is a simple but powerful convention for maintaining a history of pipeline runs without overwriting previous results. datetime.date.today().strftime('%Y%m%d') produces a lexicographically sortable date string (YYYYMMDD).
- **shutil.move() for archiving:** Moving the log file to an archive directory (rather than deleting it) is the soft-delete pattern — the log is removed from the active workspace but preserved for future auditing.

Cell 4 — ZIP Archive Operations

```
# Cell 4: Create, inspect, and extract a ZIP archive
from zipfile import ZipFile, ZIP_DEFLATED
from pathlib import Path
import os

project_root = Path('file_demo_project')

# ── Create a ZIP archive of the entire project ──────────────────
archive_path = project_root.parent / 'file_demo_project_backup.zip'

file_count = 0
total_uncompressed = 0

with ZipFile(archive_path, 'w', compression=ZIP_DEFLATED,
compresslevel=6) as zipf:
    for path in sorted(project_root.rglob('*')):
        if path.is_file():
            arcname = path.relative_to(project_root.parent)
            zipf.write(path, arcname)
            file_count += 1
            total_uncompressed += path.stat().st_size
            print(f'  + {arcname}')

archive_size = archive_path.stat().st_size
ratio = (1 - archive_size / total_uncompressed) * 100 if
total_uncompressed else 0
print(f'\nFiles archived : {file_count}')
print(f'Original size  : {total_uncompressed:,} bytes')
print(f'Archive size   : {archive_size:,} bytes')
print(f'Compression    : {ratio:.1f}%')

# ── Inspect the archive contents ────────────────────────────────
print('\n=== Archive Contents ===')
```

```
with ZipFile(archive_path) as zipf:
    for info in zipf.infolist():
        ratio_str = f'{(1-info.compress_size/info.file_size)*100:.0f}%'
if info.file_size else 'N/A'
        print(f'  {info.filename:<55} {info.file_size:>8,}B  saved:
{ratio_str}')

# ── Extract to a new location ──────────────────────────────
restore_path = Path('file_demo_restored')
with ZipFile(archive_path) as zipf:
    zipf.extractall(restore_path)
print(f'\nRestored to: {restore_path.resolve()}')
```

Explanation

This cell demonstrates the complete ZIP workflow — creation, inspection, and extraction — with a focus on the production details that matter:

- **ZIP_DEFLATED with compresslevel=6:** ZIP_DEFLATED uses zlib's DEFLATE algorithm, the standard ZIP compression method. compresslevel ranges from 1 (fastest, least compression) to 9 (slowest, best compression). Level 6 is the widely accepted balance for general use.
- **arcname = path.relative_to(project_root.parent):** The arcname argument controls the path stored inside the ZIP. Using a relative path (relative to the parent of the project root) means extracting the ZIP recreates the project_root/ directory, not dumping files into the current directory. This is the correct behaviour for archiving a named project folder.
- **Compression ratio calculation:** (1 - archive_size / total_uncompressed) * 100 computes the percentage size reduction. Observing this ratio helps calibrate compression level choices — text files (CSV, JSON, log files) typically achieve 60-80% reduction; binary files (pickle, images) often achieve little.
- **zipf.infolist():** Returns ZipInfo objects for every member of the archive, each containing the member's uncompressed size (file_size), compressed size (compress_size), filename, and timestamps. This metadata allows archive inspection without extracting a single byte.

Cell 5 — JSON Configuration Management

```
# Cell 5: JSON for configuration management — a production pattern
import json
from pathlib import Path
from typing import Any

# ── A robust config manager class ──────────────────────────
class ConfigManager:
    """
    Load, access, validate, and save project configuration from JSON.

    Supports dot-notation access for nested keys.
    """

    def __init__(self, config_path: str) -> None:
```

```
        self._path   = Path(config_path)
        self._config = self._load()

    def _load(self) -> dict:
        if not self._path.exists():
            raise FileNotFoundError(f'Config not found: {self._path}')
        with open(self._path, 'r', encoding='utf-8') as f:
            return json.load(f)

    def get(self, key: str, default: Any = None) -> Any:
        """Get a value by dot-separated key path (e.g.
'model.n_estimators')."""
        keys  = key.split('.')
        value = self._config
        for k in keys:
            if not isinstance(value, dict) or k not in value:
                return default
            value = value[k]
        return value

    def set(self, key: str, value: Any) -> None:
        """Set a nested key and save to disk immediately."""
        keys, obj = key.split('.'), self._config
        for k in keys[:-1]:
            obj = obj.setdefault(k, {})
        obj[keys[-1]] = value
        self._save()

    def _save(self) -> None:
        with open(self._path, 'w', encoding='utf-8') as f:
            json.dump(self._config, f, indent=4, ensure_ascii=False)

    def __repr__(self) -> str:
        return f'ConfigManager(path={self._path},
keys={list(self._config.keys())})'


# ── Usage ──────────────────────────────────────────────────────
cfg = ConfigManager('file_demo_project/data/raw/config.json')
print(f'Project   : {cfg.get("project")}')
print(f'Algorithm : {cfg.get("model.algorithm")}')
print(f'N Trees   : {cfg.get("model.n_estimators")}')
print(f'Threshold : {cfg.get("thresholds.passing_score")}')

# Update a nested value and auto-save
cfg.set('model.n_estimators', 200)
print(f'\nUpdated n_estimators: {cfg.get("model.n_estimators")}')
print(cfg)
```

Explanation

This cell elevates the basic json.load() / json.dump() pattern into a production-grade configuration management class — a pattern used in virtually every non-trivial ML project:

- **Dot-notation key access (get('model.algorithm')):** Nested configuration keys are accessed via a dot-separated string path, eliminating the need for chained dictionary lookups (config['model']['algorithm']) at the call site. The iterative descent through key.split('.') implements this cleanly.
- **setdefault() for nested key creation:** When setting a new nested key, obj.setdefault(k, {}) returns the existing sub-dict if it exists, or creates and returns a new empty dict if it does not. This eliminates the if k not in obj: obj[k] = {} boilerplate.
- **Auto-save on set():** Calling _save() immediately after every set() keeps the on-disk config perpetually synchronised with in-memory state. For high-frequency updates, consider batching writes; for this use case (human-controlled config), immediate persistence is correct.
- **ensure_ascii=False in json.dump():** Allows non-ASCII characters (Chinese, Arabic, accented Latin characters) to be written as their actual Unicode characters rather than escaped sequences (\u0041), making the config file human-readable for international teams.

Cell 6 — Object Persistence: Saving and Loading a Model

```
# Cell 6: Complete model persistence lifecycle
import pickle
import json
import datetime
from pathlib import Path

# ── Simulate a 'trained model' (a simple dict representing weights)
# In production this would be a sklearn estimator, PyTorch state_dict,
etc.
trained_model = {
    'weights':    [0.42, -0.17, 0.88, 0.03, -0.61],
    'bias':       0.12,
    'classes':    ['negative', 'positive'],
    'vocabulary': {'python': 0, 'data': 1, 'science': 2, 'ai': 3,
'model': 4},
}

metadata = {
    'model_name': 'SimpleSentimentClassifier',
    'version':    '1.0.0',
    'accuracy':   0.8731,
    'trained_at': datetime.datetime.now().isoformat(),
    'framework':  'custom_numpy',
}

model_dir  = Path('file_demo_project/models')
model_dir.mkdir(parents=True, exist_ok=True)

# ── Save model weights as pickle ──────────────────────────────
model_path = model_dir / 'sentiment_v1.pkl'
with open(model_path, 'wb') as f:
    pickle.dump(trained_model, f, protocol=pickle.HIGHEST_PROTOCOL)

# ── Save metadata as JSON (human-readable sidecar) ────────────
```

```
meta_path = model_dir / 'sentiment_v1_metadata.json'
meta_path.write_text(json.dumps(metadata, indent=4), encoding='utf-8')

print(f'Model saved    : {model_path}  ({model_path.stat().st_size:,}
bytes)')
print(f'Metadata saved : {meta_path}  ({meta_path.stat().st_size:,}
bytes)')

# ── Load and verify round-trip integrity ──────────────────────
with open(model_path, 'rb') as f:
    loaded_model = pickle.load(f)

loaded_meta = json.loads(meta_path.read_text(encoding='utf-8'))

print(f'\nLoaded model   : {loaded_meta["model_name"]}
v{loaded_meta["version"]}')
print(f'Accuracy       : {loaded_meta["accuracy"]:.4f}')
print(f'Trained at     : {loaded_meta["trained_at"]}')
print(f'Weights match  : {loaded_model["weights"] ==
trained_model["weights"]}')

# ── Simulate inference ────────────────────────────────────────
def predict(model, tokens):
    vocab = model['vocabulary']
    weights = model['weights']
    score = model['bias']
    for token in tokens:
        if token in vocab:
            score += weights[vocab[token]]
    return model['classes'][int(score > 0.5)]

result = predict(loaded_model, ['python', 'data', 'science'])
print(f'\nPrediction for [python, data, science]: {result}')
```

Explanation

This cell demonstrates the canonical ML model persistence pattern used in production systems — a pattern present in virtually every trained model deployment:

- **Separating weights (pickle) from metadata (JSON):** This is a fundamental architectural decision. The pickle file contains the opaque binary representation of the model. The JSON sidecar file contains human-readable metadata (version, accuracy, training date) that can be read without loading the model — enabling tooling that scans model directories to build inventories, compare versions, or display metadata in dashboards.
- **protocol=pickle.HIGHEST_PROTOCOL:** Selects the most efficient available pickle serialisation protocol (protocol 5 in Python 3.8+). Higher protocols are smaller and faster to serialise/deserialise, but may not be readable by older Python versions. HIGHEST_PROTOCOL is the correct choice for Python 3-only projects.
- **Round-trip integrity check:** Comparing loaded_model['weights'] == trained_model['weights'] after deserialisation verifies that the serialisation/deserialisation cycle produced a bit-for-bit identical object. This kind of assertion belongs in model persistence unit tests in production MLOps pipelines.

- **predict() function as inference contract:** The function only uses the loaded_model object and the input tokens — it has no dependency on any state from the training session. This is the correct design: the model object must be fully self-contained, carrying everything needed for inference within its own data structure.

Cell 7 — Building a Complete File Pipeline

```
# Cell 7: Production-grade file pipeline — end-to-end
# Simulates a real ETL pipeline: ingest raw CSVs, process, report,
archive

import csv, json, shutil, datetime
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED
from typing import Generator

# ── Configuration ──────────────────────────────────────────
PROJECT = Path('file_demo_project')
RAW_DIR  = PROJECT / 'data' / 'raw'
PROC_DIR = PROJECT / 'data' / 'processed'
RPT_DIR  = PROJECT / 'reports'
ARC_DIR  = PROJECT / 'archive'
LOG_FILE = PROJECT / 'logs' / 'pipeline.log'
LOG_FILE.parent.mkdir(parents=True, exist_ok=True)

RUN_ID = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')

def log(message: str) -> None:
    """Append a timestamped log entry to the pipeline log."""
    ts = datetime.datetime.now().isoformat(timespec='seconds')
    entry = f'{ts}  [{RUN_ID}]  {message}\n'
    with open(LOG_FILE, 'a', encoding='utf-8') as f:
        f.write(entry)
    print(entry.strip())

# ── Stage 1: Ingest raw CSVs ──────────────────────────────
def ingest_csv(filepath: Path) -> list[dict]:
    """Load and validate a student CSV file."""
    rows = []
    with open(filepath, 'r', encoding='utf-8', newline='') as f:
        for row in csv.DictReader(f):
            try:
                row['score'] = float(row['score'])
                rows.append(row)
            except ValueError:
                log(f'WARN  Skipped malformed row: {row}')
    log(f'INFO  Ingested {len(rows)} records from {filepath.name}')
    return rows

# ── Stage 2: Process (enrich) records ─────────────────────
def process_records(records: list[dict]) -> list[dict]:
    """Add computed fields to each student record."""
    for r in records:
        r['status']     = 'PASS' if r['score'] >= 70 else 'FAIL'
```

```
        r['percentile'] = round(r['score'] / 100 * 100, 1)
    return records

# ── Stage 3: Write processed CSV ──────────────────────────────
def write_processed(records: list[dict], output_path: Path) -> None:
    """Write enriched records to the processed directory."""
    output_path.parent.mkdir(parents=True, exist_ok=True)
    fields = list(records[0].keys())
    with open(output_path, 'w', encoding='utf-8', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        writer.writerows(records)
    log(f'INFO  Processed CSV written: {output_path.name}')

# ── Stage 4: Generate JSON report ─────────────────────────────
def generate_report(records: list[dict], report_path: Path) -> None:
    """Produce a summary statistics JSON report."""
    scores   = [r['score'] for r in records]
    passed   = sum(1 for r in records if r['status'] == 'PASS')
    report = {
        'run_id':        RUN_ID,
        'generated_at':  datetime.datetime.now().isoformat(),
        'total_students': len(records),
        'pass_count':    passed,
        'fail_count':    len(records) - passed,
        'pass_rate':     round(passed / len(records) * 100, 2),
        'avg_score':     round(sum(scores) / len(scores), 2),
        'max_score':     max(scores),
        'min_score':     min(scores),
    }
    report_path.parent.mkdir(parents=True, exist_ok=True)
    report_path.write_text(json.dumps(report, indent=4), encoding='utf-
8')
    log(f'INFO  Report written: {report_path.name}')

# ── Stage 5: Archive outputs ──────────────────────────────────
def archive_run(run_id: str, sources: list[Path], archive_dir: Path) ->
Path:
    """ZIP all output files from this run into a dated archive."""
    archive_dir.mkdir(parents=True, exist_ok=True)
    zip_path = archive_dir / f'run_{run_id}.zip'
    with ZipFile(zip_path, 'w', compression=ZIP_DEFLATED) as zf:
        for src in sources:
            if src.exists():
                zf.write(src, src.name)
    log(f'INFO  Archived {len(sources)} files to {zip_path.name}')
    return zip_path

# ── Run the pipeline ──────────────────────────────────────────
log('INFO  Pipeline started')

raw_files = list(RAW_DIR.glob('*.csv'))
log(f'INFO  Found {len(raw_files)} raw CSV file(s)')

all_records = []
for raw_file in raw_files:
```

```
    all_records.extend(ingest_csv(raw_file))

processed     = process_records(all_records)
processed_csv = PROC_DIR / f'students_processed_{RUN_ID}.csv'
report_json   = RPT_DIR  / f'report_{RUN_ID}.json'

write_processed(processed, processed_csv)
generate_report(processed, report_json)

archive = archive_run(RUN_ID, [processed_csv, report_json, LOG_FILE],
ARC_DIR)

log('INFO  Pipeline complete')
print(f'\nFinal archive: {archive}')
```

Explanation

This capstone cell assembles every concept from the chapter into a complete, five-stage Extract-Transform-Load (ETL) pipeline — the core architectural pattern of data engineering. Each function represents one stage of the pipeline, and each stage demonstrates a distinct file operation skill:

- **log() function:** A lightweight structured logger that appends ISO-timestamped entries to a log file. The RUN_ID (a datetime-derived string) is included in every log entry, enabling post-hoc filtering of all entries from a specific pipeline run. The open(LOG_FILE, 'a') mode appends without overwriting previous runs.
- **ingest_csv() with error handling:** The try/except inside the row-by-row loop catches malformed data on individual rows without aborting the entire ingestion. Bad rows are logged as warnings and skipped. This is the correct production pattern: fail gracefully on bad data, never silently ignore it.
- **process_records() — pure transformation:** A pure function (no file I/O, no side effects) that enriches each record in-memory. Separating computation from I/O makes this stage trivially unit-testable: just pass a list of dicts and assert on the output.
- **generate_report() — derived statistics:** Computes aggregate statistics over all processed records and serialises them to a JSON report — a pattern common in MLOps for experiment tracking and audit reporting.
- **archive_run() — immutable output packaging:** Zipping all outputs from a pipeline run into a single dated archive creates an immutable, self-contained snapshot. This is the basis of data versioning in simpler pipelines and aligns with DVC (Data Version Control) conventions.

Chapter 11: Summary — File Operations Reference

This chapter has covered the complete Python file handling landscape, from low-level discovery operations through high-level serialisation patterns. The skills covered here are directly applicable to every data pipeline, model training script, API server, and automation workflow you will build in this series.

> *"Data science is 80% data preparation — and data preparation is mostly file operations done right." — A production ML engineer's field note*

Topic	Key Skill	Primary Tools
File Discovery	List and filter files by name and pattern	os.listdir, fnmatch, glob, pathlib.rglob
File Attributes	Query size, timestamps, type	os.stat, pathlib.stat()
Directory Traversal	Recursively walk directory trees	os.walk, pathlib.rglob
Copy / Move / Rename	Manage files across directories	shutil.copy2, shutil.move, Path.rename
Delete	Remove files safely	Path.unlink, shutil.rmtree
ZIP Archives	Create, inspect, extract archives	zipfile.ZipFile, shutil.make_archive
Text Files	Read and write plain text	open() with encoding='utf-8'
CSV Files	Read and write tabular data	csv.DictReader, csv.DictWriter
JSON Files	Serialise/deserialise structured data	json.load, json.dump, json.loads, json.dumps
XML Files	Parse hierarchical documents	xml.etree.ElementTree
Object Persistence	Serialise Python objects to disk	pickle, joblib
Modern Path API	Cross-platform path manipulation	pathlib.Path